

List Induction (Part 1: Structural Induction Principles)

Lecture Notes Transcription

Course: Interactive Theorem Proving (7CCSACTP)

1. Induction on Lists (Structural Induction)

→ **Context:**

- Introduction to recursion on lists
- Structural induction principle on finite lists

→ **Intuition & Analogy:**

- **Natural numbers:** Built via finite applications of Suc .
- **Lists:** Any list is constructed with a finite sequence of Cons ($\#$) ending in $[]$:

$$x_1 \# x_2 \# \dots \# x_n \# []$$

- To prove $P(xs)$ holds for all lists xs :
 - **goal₁ (Base Case):** $P([])$
 - **goal₂ (Step Case):** Assuming $P(xs)$, prove $P(x \# xs)$ for any x .

Formal Inference Rules:

- **Natural Numbers:**

$$\frac{P(0) \quad P(m) \implies P(m+1)}{P(n)}$$

- **Lists:**

$$\frac{P([]) \quad P(y) \implies \forall y. P(y \# xs)}{P(xs)}$$

2. Worked Example: Associativity of List Append

Definition of Append (@):

$$\text{append } [] \ ys = ys \quad \text{append } (x \# xs) \ ys = x \# (\text{append } xs \ ys)$$

Theorem: $(xs @ ys) @ zs = xs @ (ys @ zs)$ (Induction on xs)

Base Case ($xs = []$):

- **goal ::** $([] @ ys) @ zs = [] @ (ys @ zs)$
- **Proof steps:**
 1. $([] @ ys) @ zs = ys @ zs$ (by *append.simps*)
 2. $\dots = [] @ (ys @ zs)$ (by *append.simps*)
- **finally** $([] @ ys) @ zs = [] @ (ys @ zs)$ ✓

Step Case ($xs \rightarrow x \# xs$):

- **Assume I.H.:** $(xs @ ys) @ zs = xs @ (ys @ zs)$
- **goal ::** $((x \# xs) @ ys) @ zs = (x \# xs) @ (ys @ zs)$
- **Proof steps:**
 1. $((x \# xs) @ ys) @ zs = (x \# (xs @ ys)) @ zs$ (by *append.simps*)
 2. $\dots = x \# ((xs @ ys) @ zs)$ (by *append.simps*)
 3. $\dots = x \# (xs @ (ys @ zs))$ (by I.H.)
 4. $\dots = (x \# xs) @ (ys @ zs)$ (by *append.simps*)
- **finally** $((x \# xs) @ ys) @ zs = (x \# xs) @ (ys @ zs)$ ✓